

Bloc 2 TD 2 - TDD PHP - Pierre Feuille Ciseaux

Contexte	2
Organisation du code	2
Les premiers tests avec PHPUnit	5
Un peu de refactoring	9
Et un dernier test avec PHPUnit	13
Aller + loin	17
Résoudre le challenge	20
Fichiers PHP finaux	21
Conclusion	21

Contexte

Ce TD va nous faire coder le challenge **Pierre Feuille Ciseaux** en **TDD** (Test Driven Development) avec **PHPUnit** : on écrit d'abord les tests, on les voit échouer, puis on code jusqu'à ce qu'ils passent.

Le système envoie la liste des coups de l'adversaire sous la forme d'une chaîne de caractères (**P** pour Pierre, **F** pour Feuille, **C** pour Ciseaux) et il faut renvoyer les coups qui gagnent à chaque fois. Par exemple pour **PFCP** il faut renvoyer **FCPF**.

Le TD s'appuie sur la sandbox de **Tainix** mais je n'ai pas de compte, j'ai donc recréé la même organisation en local et une petite classe qui **simule** le challenge (voir la partie "Résoudre le challenge"). Le projet est disponible dans le portfolio.

Organisation du code

Je reprends la même organisation que dans le TD : les dossiers

TP2/PIERRE_FEUILLE_CISEAUX pour le code et

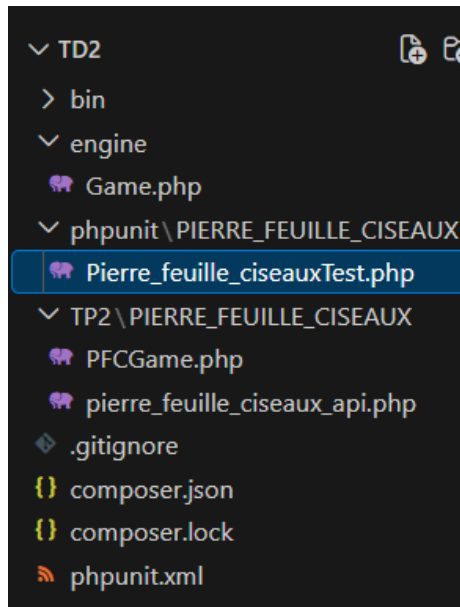
phpunit/PIERRE_FEUILLE_CISEAUX pour les tests. Dans le **composer.json**, le

namespace **TP2** est relié au dossier TP2 et le dossier vendor est dans **bin/vendor**

comme dans la commande du TD. On installe PHPUnit avec **composer install** :

```
- Installing sebastian/complexity (4.0.1): Extracting archive
- Installing sebastian/code-unit-reverse-lookup (4.0.1): Extracting archive
- Installing phpunit/php-code-coverage (11.0.12): Extracting archive
- Installing phar-io/version (3.2.1): Extracting archive
- Installing phar-io/manifest (2.0.4): Extracting archive
- Installing myclabs/deep-copy (1.14.0): Extracting archive
- Installing phpunit/phpunit (11.5.57): Extracting archive
5 package suggestions were added by new dependencies, use `composer suggest` to see details.
Generating autoload files
25 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
PS C:\xampp\htdocs\public_html\TDD\TD2> █
```

L'arborescence du projet (le dossier **engine** contient la classe qui simule le challenge) :



Dans **PFCGame.php** je crée la classe avec ses 2 méthodes, **play** qui renvoie le coup à jouer contre un coup donné et **party** qui renvoie tous les coups à jouer. Les **return ''** sont temporaires, ils permettent de respecter le typage de sortie :

```
1  <?php
2
3  declare(strict_types=1);
4
5  namespace TP2\PIERRE_FEUILLE_CISEAUX;
6
7  final class PFCGame
8  {
9      public function play(string $play): string
10     {
11         return '';
12     }
13
14     public function party(string $plays): string
15     {
16         return '';
17     }
18 }
```

Dans **Pierre_feuille_ciseauxTest.php**, j'appelle la classe PFCGame et je la charge dans la méthode **setUp()** pour l'avoir dans tous les tests (code du TD) :

```
1  <?php
2
3  use PHPUnit\Framework\TestCase;
4  use tp2\PIERRE_FEUILLE_CISEAUX\PFCGame;
5
6  final class Pierre_feuille_ciseauxTest extends TestCase
7  {
8      public function setUp(): void
9      {
10         parent::setUp();
11         $this->PFCGame = new PFCGame;
12     }
13 }
14
```

Les premiers tests avec PHPUnit

Q1 Écrivez les 3 tests :

Un test par coup : si on joue **F** la méthode doit renvoyer **C**, si on joue **C** elle doit renvoyer **P** et si on joue **P** elle doit renvoyer **F**. On utilise **assertEquals** qui vérifie que c'est égal :

```
6 final class Pierre_feuille_ciseauxTest extends TestCase
7 {
8     public function setUp(): void
9     {
10         parent::setUp();
11         $this->PFCGame = new PFCGame;
12     }
13
14     public function test_feuille_battue_par_ciseaux(): void
15     {
16         $this->assertEquals('C', $this->PFCGame->play('F'));
17     }
18
19     public function test_ciseaux_battus_par_pierre(): void
20     {
21         $this->assertEquals('P', $this->PFCGame->play('C'));
22     }
23
24     public function test_pierre_battue_par_feuille(): void
25     {
26         $this->assertEquals('F', $this->PFCGame->play('P'));
27     }
28 }
```

On lance les tests (la commande du TD est `./bin/vendor/phpunit`, mais le fichier est dans `bin\vendor\bin\phpunit`) :

```
PS C:\xampp\htdocs\public_html\TDD\TD2> .\bin\vendor\bin\phpunit --testsuite PHPUnit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD2\phpunit.xml

EEE                                                    3 / 3 (100%)

Time: 00:00.047, Memory: 8.00 MB

There were 3 errors:

1) Pierre_feuille_ciseauxTest::test_feuille_battue_par_ciseaux
Error: Class "tp2\PIERRE_FEUILLE_CISEAUX\PFCGame" not found

C:\xampp\htdocs\public_html\TDD\TD2\phpunit\PIERRE_FEUILLE_CISEAUX\Pierre_feuille_ciseauxTest.php:11

2) Pierre_feuille_ciseauxTest::test_ciseaux_battus_par_pierre
Error: Class "tp2\PIERRE_FEUILLE_CISEAUX\PFCGame" not found

C:\xampp\htdocs\public_html\TDD\TD2\phpunit\PIERRE_FEUILLE_CISEAUX\Pierre_feuille_ciseauxTest.php:11

3) Pierre_feuille_ciseauxTest::test_pierre_battue_par_feuille
Error: Class "tp2\PIERRE_FEUILLE_CISEAUX\PFCGame" not found

C:\xampp\htdocs\public_html\TDD\TD2\phpunit\PIERRE_FEUILLE_CISEAUX\Pierre_feuille_ciseauxTest.php:11

ERRORS!
Tests: 3, Assertions: 0, Errors: 3.
```

Erreur, la classe `tp2\PIERRE_FEUILLE_CISEAUX\PFCGame` n'est pas trouvée. Dans le TD le namespace est écrit en minuscule (`tp2`) mais le dossier et le namespace de la classe sont en majuscule (`TP2`), et l'autoload de Composer fait la différence entre majuscules et minuscules. Je corrige le `use` et je relance :

```
3) Pierre_feuille_ciseauxTest::test_pierre_battue_par_feuille
Failed asserting that two strings are equal.
--- Expected
+++ Actual
@@ @@
-'F'
+' '

C:\xampp\htdocs\public_html\TDD\TD2\phpunit\PIERRE_FEUILLE_CISEAUX\Pierre_feuille_ciseauxTest.php:26

FAILURES!
Tests: 3, Assertions: 3, Failures: 3, Deprecations: 1.
```

Cette fois les 3 tests échouent normalement car play n'est pas codée. Par contre il y a une **dépréciation** en plus par rapport au TD, on l'affiche avec **--display-deprecations** :

```
3 tests triggered 1 PHP deprecation:

1) C:\xampp\htdocs\public_html\TDD\TD2\phpunit\PIERRE_FEUILLE_CISEAUX\Pierre_feuille_ciseauxTest.php:11
Creation of dynamic property Pierre_feuille_ciseauxTest::$PFCGame is deprecated

Triggered by:

* Pierre_feuille_ciseauxTest::test_ciseaux_battus_par_pierre
C:\xampp\htdocs\public_html\TDD\TD2\phpunit\PIERRE_FEUILLE_CISEAUX\Pierre_feuille_ciseauxTest.php:19
```

Depuis **PHP 8.2**, créer une propriété qui n'est pas déclarée dans la classe (**\$this->PFCGame**) est déprécié. On la déclare dans la classe de test :

```
1  <?php
2
3  use PHPUnit\Framework\TestCase;
4  use TP2\PIERRE_FEUILLE_CISEAUX\PFCGame;
5
6  final class Pierre_feuille_ciseauxTest extends TestCase
7  {
8      private PFCGame $PFCGame;
9
10     public function setUp(): void
11     {
12         parent::setUp();
13         $this->PFCGame = new PFCGame;
14     }
15
16     public function test_feuille_battus_pier_pierre()
```

```
FAILURES!
Tests: 3, Assertions: 3, Failures: 3.
PS C:\xampp\htdocs\public_html\TDD\TD2>
```

On a maintenant exactement le résultat du TD : 3 tests, 3 échecs, ce qui est normal puisque la méthode play n'est pas encore codée. On code play avec 3 if, comme dans le TD :

```
7  final class PFCGame
8  {
9      public function play(string $play): string
10     {
11         if ($play === 'P') {
12             return 'F';
13         }
14
15         if ($play === 'F') {
16             return 'C';
17         }
18
19         if ($play === 'C') {
20             return 'P';
21         }
22     }
23
24     public function party(string $plays): string
25     {
26         return '';
27     }
28 }
```

```
... 3 / 3 (100%)
Time: 00:00.011, Memory: 8.00 MB
OK (3 tests, 3 assertions)
```

Les 3 tests passent.

Q2 Proposez une autre façon :

On peut utiliser un **match** (depuis PHP 8). Il compare la valeur avec **===** comme nos if, et il renvoie directement la valeur, donc pas besoin de répéter les return :

```
6
7  final class PFCGame
8  {
9      public function play(string $play): string
10     {
11         return match($play) {
12             'P' => 'F',
13             'F' => 'C',
14             'C' => 'P',
15         };
16     }
17
18     public function party(string $plays): string
19     {
20         return '';
21     }
22 }
```

```
PS C:\xampp\htdocs\public_html\TDD\TD2> .\bin\vendor\bin\phpunit --testsuite PHPUnit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD2\phpunit.xml

...                                                    3 / 3 (100%)

Time: 00:00.011, Memory: 8.00 MB

OK (3 tests, 3 assertions)
PS C:\xampp\htdocs\public_html\TDD\TD2>
```

Toujours vert. On aurait pu aussi faire un **switch**, mais il compare avec **==** et il faut un break ou un return dans chaque case, le match est plus court.

Un peu de refactoring

Comme les tests verrouillent le bon fonctionnement de play, on peut réécrire le code. On remplace les chaînes de caractères par des **constantes de classe** : si on tape "CC" au lieu de "C" il n'y a pas d'erreur, alors que si on tape self::CISEAU (sans X) PHP renvoie une erreur.

J'avais déjà commencé cette version dans mon fichier **PIERRE_FEUILLE_CISEAUX.PHP**, je reprends donc son code (la méthode party reste à return "" pour l'instant) et je relance les tests :

```
PS C:\xampp\htdocs\public_html\TDD\TD2> .\bin\vendor\bin\phpunit --testsuite PHPUnit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD2\phpunit.xml

PHP Fatal error:  strict types declaration must be the very first statement in the script in C:\xampp\htdocs\public_html\TDD\TD2\PIERRE_FEUILLE_CISEAUX\PFCGame.php on line 19

Fatal error: strict types declaration must be the very first statement in the script in C:\xampp\htdocs\public_html\TDD\TD2\PIERRE_FEUILLE_CISEAUX\PFCGame.php on line 19
```

Fatal error : dans mon fichier, la consigne du challenge était en commentaire avant le `<?php` :

```
11
12 Pierre : P
13 Feuille : F
14 Ciseaux : C
15 Par exemple, le système va renvoyer PFCP. 4 coups donc : Pierre, Feuille, Cise
16 */
17
18 <?php
19 declare(strict_types=1);
20
21 namespace TP2\PIERRE_FEUILLE_CISEAUX;
22
23 final class PFCGame
24 {
25     private const PIERRE = 'P';
26     private const FEUILLE = 'F';
27     private const CISEAUX = 'C';
```

Tout ce qui est avant le `<?php` est du texte envoyé tel quel (comme du HTML), donc **declare(strict_types=1)** n'est plus la première instruction du fichier, alors que c'est obligatoire. Je déplace la consigne dans un commentaire **dans** le PHP, au-dessus de la classe :

```
1 <?php
2
3 declare(strict_types=1);
4
5 namespace TP2\PIERRE_FEUILLE_CISEAUX;
6
7 /**
8  * Mission
9  * Le système renvoie la liste des coups de ton adversaire sous la forme d'une chaîne
10  *
11  * Ici, on ne rigole pas, ce n'est pas le premier à 3 qui gagne, il faut gagner tous l
12  *
13  * Si ton adversaire fait Pierre, tu dois faire Feuille, s'il fait Feuille, tu dois fa
```

```

23 final class PFCGame
24 {
25     private const PIERRE = 'P';
26     private const FEUILLE = 'F';
27     private const CISEAUX = 'C';
28
29     public function play(string $play): string
30     {
31         if ($play === self::PIERRE) {
32             return self::FEUILLE;
33         }
34
35         if ($play === self::FEUILLE) {
36             return self::CISEAUX;
37         }
38
39         if ($play === self::CISEAUX) {
40             return self::PIERRE;
41         }
42     }
43
44     public function party(string $plays): string
45     {
46         return '';
47     }
48 }

```

```

PS C:\xampp\htdocs\public_html\TDD\TD2> .\bin\vendor\bin\phpunit --testsuite PHPUnit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD2\phpunit.xml

...                                                  3 / 3 (100%)

Time: 00:00.016, Memory: 8.00 MB

OK (3 tests, 3 assertions)

```

Si on relance les tests, on a toujours du vert.

Q3 Refactoriser ce code, je crois qu'on peut faire mieux que ces 3 if :

On met dans une constante un **tableau associatif** qui donne, pour chaque coup de l'adversaire, le coup qui le bat. La méthode play n'a plus qu'à lire le tableau (sélectionné) :

```
22
23 final class PFCGame
24 {
25     private const PIERRE = 'P';
26     private const FEUILLE = 'F';
27     private const CISEAUX = 'C';
28
29     private const COUPS_GAGNANTS = [
30         self::PIERRE => self::FEUILLE,
31         self::FEUILLE => self::CISEAUX,
32         self::CISEAUX => self::PIERRE,
33     ];
34
35     public function play(string $play): string
36     {
37         return self::COUPS_GAGNANTS[$play];
38     }
39
40     public function party(string $plays): string
41     {
42         return '';
43     }
44 }
```

```
PS C:\xampp\htdocs\public_html\TDD\TD2> .\bin\vendor\bin\phpunit --testsuite PHPUnit --testdox
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD2\phpunit.xml

...                                                  3 / 3 (100%)

Time: 00:00.012, Memory: 8.00 MB

Pierre feuille ciseaux
✓ Feuille battue par ciseaux
✓ Ciseaux battus par pierre
✓ Pierre battue par feuille

OK (3 tests, 3 assertions)
```

Avec **--testdox**, PHPUnit affiche le nom des tests, et ils passent tous. Si un jour il y a un nouveau coup (Puits par exemple), il suffira de rajouter une ligne dans le tableau.

Et un dernier test avec PHPUnit

Q4 Écrivez la méthode party :

On écrit d'abord le test : si l'adversaire fait **PFC**, la méthode party doit renvoyer **FCP** :

```
26     public function test_pierre_battue_par_feuille(): void
27     {
28         $this->assertEquals('F', $this->PFCGame->play('P'));
29     }
30
31     public function test_partie_3_coups(): void
32     {
33         $this->assertEquals('FCP', $this->PFCGame->party('PFC'));
34     }
35 }
36
```

```
PS C:\xampp\htdocs\public_html\TDD\TD2> .\bin\vendor\bin\phpunit --testsuite PHPUnit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD2\phpunit.xml

...F                                                4 / 4 (100%)

Time: 00:00.027, Memory: 8.00 MB

There was 1 failure:

1) Pierre_feuille_ciseauxTest::test_partie_3_coups
Failed asserting that two strings are equal.
--- Expected
+++ Actual
@@ @@
-'FCP'
+'''

C:\xampp\htdocs\public_html\TDD\TD2\phpunit\PIERRE_FEUILLE_CISEAUX\Pierre_feuille_ciseauxTest.php:33

FAILURES!
Tests: 4, Assertions: 4, Failures: 1.
```

Quelques explications sur l'affichage :

- Les **.** représentent les tests qui passent et le **F** un échec

- “Failed asserting that two strings are equal” : les 2 chaînes ne sont pas égales, on attendait 'FCP' (Expected) et on a eu '' (Actual)
- Le récapitulatif à la fin : 4 tests, 4 assertions, 1 échec

On code party, qui appelle play pour chaque caractère de la chaîne (c’est la boucle de mon fichier PIERRE_FEUILLE_CISEAUX.PHP) :

```

35     public function play(string $play): string
36     {
37         return self::COUPS_GAGNANTS[$play];
38     }
39
40     public function party(string $plays): string
41     {
42         $response = '';
43         $nbPlays = strlen($plays) - 1;
44
45         for ($i = 0; $i <= $nbPlays; $i++) {
46             $response .= $this->play($plays[$i]);
47         }
48
49         return $response;
50     }
51 }
52

```

```

PS C:\xampp\htdocs\public_html\TDD\TD2> .\bin\vendor\bin\phpunit --testsuite PHPUnit
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD2\phpunit.xml

....                                                  4 / 4 (100%)

Time: 00:00.012, Memory: 8.00 MB

OK (4 tests, 4 assertions)

```

4 tests écrits et validés avec PHPUnit. Jusqu’ici on a testé le “good path”, c’est-à-dire que tout ce qui doit bien se passer se passe bien.

Aller + loin : le “wrong path”

Qu'est-ce qui se passe si je passe une chaîne vide à la méthode `play`, ou un autre caractère que `P`, `F` ou `C` ? Pour le savoir, on écrit les tests : on veut qu'une `InvalidArgumentException` soit levée si le coup n'est pas valide (chaîne vide, “X”, ou “p” en minuscule), que `party` renvoie une chaîne vide si la partie est vide, et qu'elle lève l'exception s'il y a un coup invalide dans la partie :

```
37 public function test_play_chaine_vide(): void
38 {
39     $this->expectException(InvalidArgumentException::class);
40     $this->PFCGame->play('');
41 }
42
43 public function test_play_autre_caractere(): void
44 {
45     $this->expectException(InvalidArgumentException::class);
46     $this->PFCGame->play('X');
47 }
48
49 public function test_play_minuscule(): void
50 {
51     $this->expectException(InvalidArgumentException::class);
52     $this->PFCGame->play('p');
53 }
54
55 public function test_partie_vide(): void
56 {
57     $this->assertEquals('', $this->PFCGame->party(''));
58 }
59
60 public function test_partie_avec_coup_invalide(): void
61 {
62     $this->expectException(InvalidArgumentException::class);
63     $this->PFCGame->party('PFXC');
64 }
```

On aurait pu aussi faire + de tests pour la méthode `party`, avec 1, 2, 3, 5, 10, 20 coups : on le fait avec un **DataProvider**, une méthode qui renvoie les parties à tester avec le résultat attendu, PHPUnit lance le test une fois par ligne :

```

66     public static function parties(): array
67     {
68         return [
69             '1 coup' => ['P', 'F'],
70             '2 coups' => ['PF', 'FC'],
71             '3 coups' => ['PFC', 'FCP'],
72             '5 coups' => ['CCFPF', 'PPFCF'],
73             '10 coups' => ['PFCPCPCPCP', 'FCPCPCPCPC'],
74             '20 coups' => ['PPFFCCPCPCPPFFCCPCPC', 'FFCCPPPCPCFFCCPPPCPC'],
75         ];
76     }
77
78     #[DataProvider('parties')]
79     public function test_partie_plusieurs_coups(string $coups, string $attendu): void
80     {
81         $this->assertEquals($attendu, $this->PFCGame->party($coups));
82     }
83 }

```

On lance les tests sans toucher au code :

```

There were 4 failures:

1) Pierre_feuille_ciseauxTest::test_play_chaine_vide
Failed asserting that exception of type "TypeError" matches expected exception "InvalidArgumentException". Message was: "T
urn value must be of type string, null returned" at
C:\xampp\htdocs\public_html\TDD\TD2\TP2\PIERRE_FEUILLE_CISEAUX\PFCGame.php:37
C:\xampp\htdocs\public_html\TDD\TD2\phpunit\PIERRE_FEUILLE_CISEAUX\Pierre_feuille_ciseauxTest.php:40
.

```

```

FAILURES!
Tests: 15, Assertions: 15, Failures: 4, Warnings: 3.

```

Les parties de 1 à 20 coups et la partie vide passent déjà, mais avec un mauvais coup, PHP affiche un **warning** (la clé n'existe pas dans le tableau), le tableau renvoie null et comme `play` doit renvoyer une string, on a une **TypeError**. On rajoute une vérification au début de `play` :

```

35     public function play(string $play): string
36     {
37         if (!isset(self::$COUPS_GAGNANTS[$play])) {
38             throw new InvalidArgumentException("Coup invalide : '$play' (P, F ou C attendu)");
39         }
40
41         return self::$COUPS_GAGNANTS[$play];

```

```

PS C:\xampp\htdocs\public_html\TDD\TD2> .\bin\vendor\bin\phpunit --testsuite PHPUnit --testdox
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD2\phpunit.xml

..... 15 / 15 (100%)

Time: 00:00.015, Memory: 8.00 MB

Pierre_feuille_ciseaux
✓ Feuille battue par ciseaux
✓ Ciseaux battus par pierre
✓ Pierre battue par feuille
✓ Partie 3 coups
✓ Play chaine vide
✓ Play autre caractere
✓ Play minuscule
✓ Partie vide
✓ Partie avec coup invalide
✓ Partie plusieurs coups with 1·coup
✓ Partie plusieurs coups with 2·coups
✓ Partie plusieurs coups with 3·coups
✓ Partie plusieurs coups with 5·coups
✓ Partie plusieurs coups with 10·coups
✓ Partie plusieurs coups with 20·coups

OK (15 tests, 15 assertions)

```

15 tests, tout est vert. Maintenant un mauvais coup donne une erreur claire au lieu d'un warning.

Aller + loin

Q5 Écrivez la méthode “private” :

La méthode play n'est utilisée que dans la classe (par party), elle devrait donc être **private**. Je la passe en private et je relance les tests :

```

3) Pierre_feuille_ciseauxTest::test_pierre_battue_par_feuille
Error: Call to private method TP2\PIERRE_FEUILLE_CISEAUX\PFGame::play() from scope Pierre_feuille_ciseauxTest
C:\xampp\htdocs\public_html\TDD\TD2\phpunit\PIERRE_FEUILLE_CISEAUX\Pierre_feuille_ciseauxTest.php:29

```

ERRORS!

Tests: 15, Assertions: 12, Errors: 3, Failures: 3.

Normal, les tests appellent play directement, ce qui n'est plus possible. Pour tester quand même une méthode private, on utilise la **réflexion** de PHP :

ReflectionMethod permet de récupérer la méthode et **invoke()** de l'appeler sur notre objet. Je fais une petite méthode play() dans la classe de test et les tests l'utilisent à la place de \$this->PFCGame->play() :

```
8  {
9      private PFCGame $PFCGame;
10
11     public function setUp(): void
12     {
13         parent::setUp();
14         $this->PFCGame = new PFCGame;
15     }
16
17     private function play(string $coup): string
18     {
19         $methode = new ReflectionMethod(PFCGame::class, 'play');
20
21         return $methode->invoke($this->PFCGame, $coup);
22     }
23
24     public function test_feuille_battue_par_ciseaux(): void
25     {
26         $this->assertEquals('C', $this->play('F'));
27     }
28
29     public function test_ciseaux_battus_par_pierre(): void
30     {
31         $this->assertEquals('P', $this->play('C'));
32     }
33 }
```

Et dans PFCGame, play est maintenant private :

```
35 private function play(string $play): string
36 {
37     if (!isset(self::COUPS_GAGNANTS[$play])) {
38         throw new \InvalidArgumentException("Coup invalide : '$play' (P, F ou C attendu)");
39     }
40
41     return self::COUPS_GAGNANTS[$play];
42 }
43
44 public function party(string $plays): string
45 {
46     $response = '';
47     $nbPlays = strlen($plays) - 1;
48
49     for ($i = 0; $i <= $nbPlays; $i++) {
50         $response .= $this->play($plays[$i]);
51     }
52
53     return $response;
54 }
55 }
```

```
PS C:\xampp\htdocs\public_html\TDD\TD2> .\bin\vendor\bin\phpunit --testsuite PHPUnit
PHPUnit 11.5.7 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD2\phpunit.xml

.....
15 / 15 (100%)

Time: 00:00.014, Memory: 8.00 MB

OK (15 tests, 15 assertions)
```

Tout repasse au vert. Une autre solution aurait été de ne tester que party (la méthode publique), puisque party utilise play : si play ne marche pas, les tests de party échouent.

Résoudre le challenge

Dans le fichier **pierre_feuille_ciseaux_api.php**, on récupère les coups de l'adversaire et on appelle party, comme dans le TD :

```
1  <?php
2
3  declare(strict_types=1);
4
5  require_once __DIR__ . '/../../bin/vendor/autoload.php';
6
7  use Engine\Game;
8  use TP2\PIERRE_FEUILLE_CISEAUX\PFCGame;
9
10 $game = new Game('PIERRE_FEUILLE_CISEAUX');
11
12 $coups = $game->input()['coups'];
13
14 // CODE DU tp -----
15 $output = (new PFCGame)->party($coups);|
16
17
18
19 // REPONSE -----
20 $game->output(['data' => $output]);
21
```

N'ayant pas de compte Tainix, la classe **Engine\Game** est une simulation : elle génère 20 coups au hasard pour l'adversaire et vérifie la réponse. On ouvre la page dans le navigateur
(http://localhost/public_html/TDD/TD2/TP2/PIERRE_FEUILLE_CISEAUX/pierre_feuille_ciseaux_api.php) :

Challenge PIERRE_FEUILLE_CISEAUX (simulation locale)

Coups de l'adversaire : FPPFPCFCCCPCFFFCPPFF

Ma réponse : CFFCFPCPPFPCCCPFFCC

Félicitations ! Challenge réussi avec succès !

Challenge réussi, encore du vert !

Fichiers PHP finaux

Q6 Envoyez les fichiers finaux :

Les fichiers sont envoyés avec ce compte rendu :

- **PFCGame.php** (la classe avec les constantes, le tableau des coups gagnants, play en private et party)
- **Pierre_feuille_ciseauxTest.php** (les 15 tests)
- le dossier **projet** avec tout le projet (sans le dossier bin/vendor, il se réinstalle avec composer install)

Le résultat final des tests :

```
PS C:\xampp\htdocs\public_html\TDD\TD2> .\bin\vendor\bin\phpunit --testsuite PHPUnit --testdox
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\public_html\TDD\TD2\phpunit.xml

.....                                                    15 / 15 (100%)

Time: 00:00.015, Memory: 8.00 MB

pierre_feuille_ciseaux
✓ Feuille battue par ciseaux
✓ Ciseaux battus par pierre
✓ Pierre battue par feuille
✓ Partie 3 coups
✓ Play chaîne vide
✓ Play autre caractère
✓ Play minuscule
✓ Partie vide
✓ Partie avec coup invalide
✓ Partie plusieurs coups with 1·coup
✓ Partie plusieurs coups with 2·coups
✓ Partie plusieurs coups with 3·coups
✓ Partie plusieurs coups with 5·coups
✓ Partie plusieurs coups with 10·coups
✓ Partie plusieurs coups with 20·coups

OK (15 tests, 15 assertions)
```

Conclusion

Ce **TD** nous a fait coder le challenge Pierre Feuille Ciseaux en **TDD** avec **PHPUnit** : écrire les tests, les voir rouges, coder jusqu’au vert puis refactoriser sans risque. On a aussi testé le “wrong path” avec des exceptions, utilisé un DataProvider et testé une méthode private avec la réflexion. Les tests nous ont aussi aidés à trouver des erreurs qui n’étaient pas dans le TD (le namespace en minuscule et le commentaire avant le <?php).